

Звіт

до Лабораторної роботи №5

Успадкування в Java

Студент: Колісніченко Артем Олександрович

Група: КН-924в

Викладач: Савченко В. М.

Рік: 2026

1. Мета роботи

Метою лабораторної роботи є отримання практичних навичок побудови ієрархії класів у мові програмування Java та використання механізму успадкування.

У ході роботи необхідно побудувати ієрархію класів глибиною 3–4 рівні без застосування абстрактних класів та інтерфейсів, показати використання `extends`, `super(...)`, `@Override` і `final`, створити UML-діаграму класів та виконати JOL-аналіз одного з кінцевих класів.

2. Опис предметної галузі

У даній лабораторній роботі використовується предметна галузь міської інфраструктури. Для варіанта 5 потрібно побудувати ієрархію класів, пов'язану з будівлями.

У роботі базовим поняттям є будівля. Звичайна будівля може бути громадською, громадська будівля може бути навчальною, а навчальна будівля може бути університетським корпусом.

Також на рівні громадської будівлі було додано окреме відгалуження — медичну будівлю.

Основна ієрархія:

Building -> PublicBuilding -> EducationalBuilding ->
UniversityCampusBuilding

Додаткове відгалуження:

PublicBuilding -> MedicalBuilding

3. Побудова ієрархії класів

3.1 Базовий клас Building

Клас Building є базовим класом у створеній ієрархії. Він описує звичайну будівлю та містить основні характеристики: адресу та рік побудови.

Власні поля класу:

- address — адреса будівлі;
- yearBuilt — рік побудови.

Також у класі є метод description(), який повертає текстовий опис об'єкта.

3.2 Клас PublicBuilding

Клас PublicBuilding успадковує клас Building за допомогою ключового слова extends. Він описує громадську будівлю.

Такий зв'язок є коректним, тому що громадська будівля є різновидом звичайної будівлі.

Власне поле класу:

- visitorCapacity — кількість відвідувачів, яку може прийняти будівля.

У конструкторі використано виклик super(...), щоб передати адресу та рік побудови до конструктора базового класу.

3.3 Клас EducationalBuilding

Клас EducationalBuilding успадковує PublicBuilding. Він описує навчальну будівлю.

Навчальна будівля є різновидом громадської будівлі, тому такий зв'язок є логічним для цієї предметної області.

Власне поле класу:

- `classroomsCount` — кількість аудиторій.

Метод `description()` перевизначено за допомогою `@Override`, щоб додати інформацію про кількість аудиторій.

3.4 Клас `UniversityCampusBuilding`

Клас `UniversityCampusBuilding` успадковує `EducationalBuilding`. Він є кінцевим класом основної ієрархії та описує університетський корпус.

Власне поле класу:

- `facultyName` — назва факультету.

Клас оголошено як `final`. Це означає, що від нього не можна створювати нові дочірні класи. Таким чином у роботі показано використання ключового слова `final` у контексті ієрархії.

3.5 Клас `MedicalBuilding`

Клас `MedicalBuilding` є додатковим відгалуженням від `PublicBuilding`. Він описує медичну будівлю.

Медична будівля також є громадською будівлею, тому успадкування від `PublicBuilding` є коректним.

Власне поле класу:

- `doctorsCount` — кількість лікарів.

4. Обґрунтування коректності ієрархії

Побудована ієрархія є семантично коректною, тому що між класами існує зв'язок «є різновидом».

`PublicBuilding` є різновидом `Building`, тому що громадська будівля також є будівлею.

`EducationalBuilding` є різновидом `PublicBuilding`, тому що навчальна будівля є громадською будівлею.

`UniversityCampusBuilding` є різновидом `EducationalBuilding`, тому що університетський корпус є навчальною будівлею.

MedicalBuilding є різновидом PublicBuilding, тому що медична будівля також належить до громадських будівель.

Отже, успадкування використовується не просто для повторного використання коду, а для відображення реальних зв'язків предметної області.

5. Власний стан кожного класу

Усі класи ієрархії мають власний стан. Це важливо, тому що клас, який не додає нових полів, був би штучним проміжним рівнем.

Клас	Власний стан
Building	address, yearBuilt
PublicBuilding	visitorCapacity
EducationalBuilding	classroomsCount
UniversityCampusBuilding	facultyName
MedicalBuilding	doctorsCount

6. Використані механізми Java

У створеній ієрархії використано основні механізми Java, пов'язані з успадкуванням.

Меха нізм	Приклад використання
--------------	----------------------

extends	Успадкування PublicBuilding від Building, EducationalBuilding від PublicBuilding, UniversityCampusBuilding від EducationalBuilding.
super(...)	Виклик конструктора батьківського класу для ініціалізації успадкованих полів.
@Override	Перевизначення методу description() у класах-нащадках.
final	Клас UniversityCampusBuilding оголошено як final.

Наприклад, у класі EducationalBuilding використано виклик:

```
super(address, yearBuilt, visitorCapacity);
```

Цей виклик передає параметри до конструктора батьківського класу PublicBuilding.

7. Аналіз моделі з лабораторної роботи №2

У лабораторній роботі №2 використовувалась модель спортивного клубу. Основними класами цієї моделі були:

- SportsClub;
- Student;
- Coach;
- SportSection;
- Schedule;
- Enrollment.

Клас SportsClub описує спортивний клуб. Він містить список секцій, список студентів та список записів на секції. Тому цей клас не є різновидом студента, тренера або секції. Він лише використовує інші класи у своїй роботі.

Клас SportSection описує спортивну секцію. Він має назву, тренера та розклад. Це також не є різновидом тренера або розкладу, тому тут краще використовувати зв'язок «має», а не успадкування.

Клас Enrollment описує запис студента на секцію. Він пов'язує студента, секцію та дату запису. Цей клас не є різновидом ні студента, ні секції.

Клас Schedule описує розклад занять. Він також не є різновидом секції, а тільки використовується нею.

Найбільш логічне узагальнення у цій моделі можна зробити для класів Student і Coach, тому що обидва вони описують людей.

Person -> Student

Person -> Coach

Спільний стан базового класу Person:

- fullName.

Власний стан похідних класів:

- Student може мати групу або курс;
- Coach має sportType, тобто вид спорту.

Таке узагальнення є доречним, тому що студент і тренер справді є різновидами людини. Для класів SportsClub, SportSection, Schedule і Enrollment успадкування краще не вводити, тому що між ними немає коректного зв'язку «є різновидом».

8. Узагальнення бібліотечної моделі

У додатковому завданні також потрібно розглянути спрощену бібліотечну модель. Для неї можна ввести базовий клас LibraryItem.

Від нього можуть успадковуватися:

- Book;
- Magazine;
- Newspaper;
- EBook.

LibraryItem -> Book

LibraryItem -> Magazine

LibraryItem -> Newspaper

LibraryItem -> EBook

Спільний стан базового класу LibraryItem:

- inventoryNumber — інвентарний номер;
- title — назва;
- available — ознака доступності.

Клас	Власний стан
Book	isbn, author
Magazine	issueNumber
Newspaper	date
EBook	fileFormat, fileSizeMb

Таке узагальнення є доречним, тому що книга, журнал, газета та електронна книга є різновидами бібліотечної одиниці.

9. UML-діаграма класів

Для створеної ієрархії було побудовано UML-діаграму класів. На ній показано всі класи та зв'язки успадкування між ними.

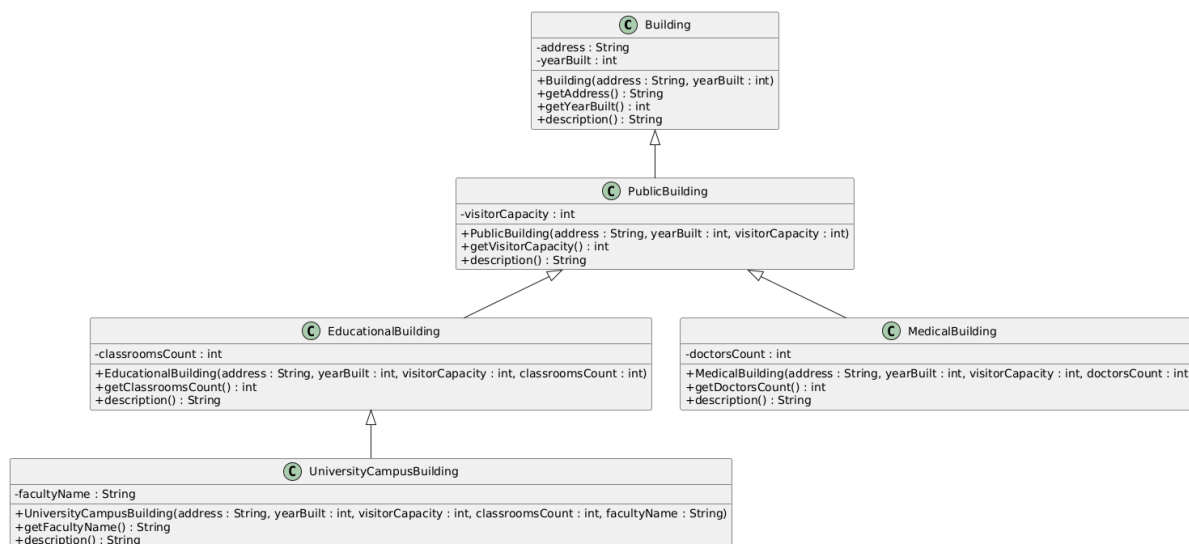


Рисунок 1 — UML-діаграма ієрархії класів

10. Демонстрація роботи програми

Для демонстрації роботи було створено клас Main. У ньому створюються об'єкти різних рівнів ієрархії, після чого для кожного об'єкта викликається метод `description()`.

Приклад результату роботи програми:

```

Building [address=Харків, проспект Науки, 10, yearBuilt=1980]
Building [address=Харків, вулиця Сумська, 20, yearBuilt=1995], visitorCapacity=300
Building [address=Харків, вулиця Кирпичова, 2, yearBuilt=1965], visitorCapacity=500, classroomsCount=25
Building [address=Харків, вулиця Кирпичова, 2, yearBuilt=1965], visitorCapacity=500, classroomsCount=25, facultyName=Комп'ютерні науки
Building [address=Харків, вулиця Пушкінська, 15, yearBuilt=2005], visitorCapacity=200, doctorsCount=40

Process finished with exit code 0
  
```

Рисунок 2 — приклад роботи програми

11. JOL-аналіз

Для JOL-дослідження було створено клас JolMain. У ньому створюється об'єкт кінцевого класу `UniversityCampusBuilding`.

Цей клас було обрано для аналізу, тому що він знаходиться в кінці основної ієрархії та містить як власний стан, так і успадкований стан усіх попередніх класів.

Результат JOL-аналізу:

У виводі видно, що один об'єкт `UniversityCampusBuilding` містить поля всіх рівнів ієрархії:

- `Building.yearBuilt`;
- `Building.address`;
- `PublicBuilding.visitorCapacity`;
- `EducationalBuilding.classroomsCount`;
- `UniversityCampusBuilding.facultyName`.

Розмір екземпляра становить 32 bytes. Це показує, що об'єкт дочірнього класу не є набором окремих вкладених об'єктів. Він є одним об'єктом у пам'яті JVM, який містить успадковані та власні поля.

Тут вставити скриншот JOL-виводу

```
ua.khpi.oop.lab05.UniversityCampusBuilding object internals:
OFF  SZ                TYPE DESCRIPTION                               VALUE
 0   8                (object header: mark)                        0x0000000000000001 (non-biasable; age: 0)
 8   4                (object header: class)                        0x01043240
12   4                int Building.yearBuilt                        1965
16   4  java.lang.String Building.address                          (object)
20   4                int PublicBuilding.visitorCapacity          500
24   4                int EducationalBuilding.classroomsCount      25
28   4  java.lang.String UniversityCampusBuilding.facultyName      (object)
Instance size: 32 bytes
Space losses: 0 bytes internal + 0 bytes external = 0 bytes total
```

Рисунок 3 — результат JOL-аналізу класу `UniversityCampusBuilding`

12. Тестування

Для перевірки роботи програми були створені тести `TestNG`.

Тестові класи:

- `BuildingTest`;
- `PublicBuildingTest`;
- `EducationalBuildingTest`;
- `UniversityCampusBuildingTest`;
- `MedicalBuildingTest`.

Тести перевіряють правильність роботи конструкторів, збереження власного стану класів, доступ до успадкованих полів та роботу методу `description()`.

```
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.420 s -- in TestSuite
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
```

Рисунок 4 — результат виконання тестів

13. Висновок

У ході виконання лабораторної роботи було побудовано ієрархію класів для варіанта 5 — міська інфраструктура.

Основна ієрархія має чотири рівні:

Building -> PublicBuilding -> EducationalBuilding ->
UniversityCampusBuilding

Також було додано відгалуження:

PublicBuilding -> MedicalBuilding

У роботі не використовувалися абстрактні класи та інтерфейси. Усі класи мають власний стан, тому ієрархія не є штучною.

Було продемонстровано використання механізмів `extends`, `super(...)`, `@Override` і `final`.

JOL-аналіз показав, що об'єкт кінцевого класу містить поля всіх рівнів ієрархії, але при цьому залишається одним об'єктом у пам'яті JVM.

Також було проаналізовано модель з лабораторної роботи №2. Було встановлено, що коректне узагальнення через зв'язок «є різновидом» можна зробити для `Student` і `Coach` через базовий клас `Person`, але для `SportsClub`, `SportSection`, `Schedule` і `Enrollment` успадкування краще не використовувати.